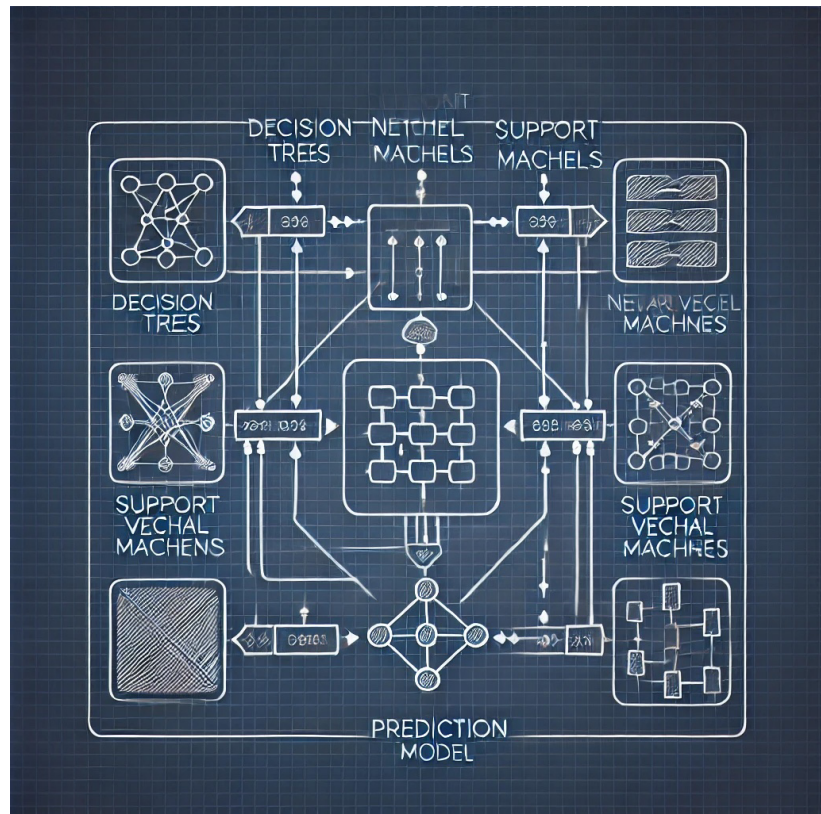




COMP47590

Advanced Machine Learning

Assignment 1: Benchmarking Ensemble Methods

Introduction

The ensemble is one of the most impactful ideas in machine learning. One interesting idea that has emerged in recent years is to use ensembling instead of hyper-parameter tuning. So, rather than tuning hyper parameters for a single model, an ensemble is created containing many models using different hyper-parameter combinations.

Tasks

Perform the following tasks:

1. In Python, inheriting from the appropriate **scikit-learn** base classes, define a new **HyperParamEnsembleClassifier** model that implements an ensemble models using a single base estimator type, and different hyperparameter combinations for each model in the ensemble. Note:
 - The model should work for multiclass classification problems and accept numeric inputs.
 - The class should implement training the model by training each of its member models (implemented in the **fit** method) and making predictions (implemented through both the **predict** and **predict_proba** methods) using voting.
 - Parameters passed to the model constructor should include the base estimator to use, a Python dictionary defining the hyperparameter ranges to select from (following the style used by scikit-learn for hyper-parameter tuning), and the number of estimators to include. number of **epochs** that the model should be trained for, and the **learning rate** used in gradient descent.
2. Design a suitable evaluation experiment to evaluate the performance of this model.
 - Choose an appropriate performance measure to use.
 - Choose an appropriate set of at least 4 algorithms to benchmark the new ensemble method against.
 - Choose an appropriate set of at least 5 datasets to use in this benchmark.
3. Complete the benchmark experiment following the best practice discussed in class and output appropriate results.
4. Write a short (less than 400 words) reflection on the results of the experiment and the experience of running it.

Notes

The following notes may be useful:

- **Can I Use Scikit-Learn and Other Python Packages?** One of the goals of this assignment is to gain experience writing an original machine learning algorithm implementation rather than simply using existing packages. We recognise, however, re-implementing things needlessly is often of limited value. So, for this assignment we seek to strike a balance. You must write your own implementation the **HyperParamClassifier** class, but you can use existing scikit-learn implementations for performing tasks like base-estimators, cross validation, grid searches, and measuring performance. Similarly, all scikit-learn models are built on top of **numpy** and it provides many useful utility functions. If in doubt about the use of a package please just ask.
- **What Hardware Should I Use?** It should be possible to complete the computation needed for this experiment . If you are having problems, however, you should consider using Google Colab¹, a great online platform for machine learning.
- **Plagiarism.** Without looking too hard online you can find plenty of ensemble implementations that you could copy and submit for this assignment. Similarly, you could just ask ChatGPT/Claude, etc to write this for you. **Don't do this!** Your submission should be your, or your team's (see below), own work. If there is any uncertainty you may be interviewed about your submission and must be able to explain every line of code in it. If you are unsure about anything in this regard just ask.
- **Teams.** You can complete this assignment as an individual or as a team of up to three. Submissions will be graded in the same way whether submitted by an individual or by a group, and both members of a team will receive the same grade.

¹ Google colab <https://colab.research.google.com>

Submission

The key submission details for the assignment are as follows:

- **Submission date:** Monday 10th March 2025 before 23:59.
- **Submission method:** Submissions should be made through the module Brightspace site.
- **Submission format:** Submissions should compose a zip file containing the following:
 - a completed Jupyter notebook provided in .ipynb format (the Jupyter notebook and code should include adequate comments or markdown commentary to understand your implementation);
 - a HTML export of your Jupyter notebook after execution that contains all output;
 - any other files required to execute your code (datasets do not need to be included but links to easy downloads for them should be included in the Jupyter notebook).
- **Group submission:** For group submissions only one member of the group should submit. Names and student numbers should be marked clearly at the top of the Jupyter notebook.
- **Late submissions:** Late submissions will be penalised at 5% penalty per day.

Marking

Marking of tasks will be based on the following weighting.

- Task 1 45% HyperParamClassifier Implementation
- Task 2 20% Evaluation Experiment Design
- Task 3 25% Evaluation Experiment Execution
- Task 4 10% Reflection Text